Architecture

This document is the deep architectural reference for the **VIP Connect External Campaigns** platform. Pair it with `TECHNICAL_HANDOFF.md` for environment specifics and `DATA_DICTIONARY.md` for record schemas.

1. C4 Container Diagram

```
flowchart TB
    subgraph Browser["Operator Browser"]
        UI[VIP Admin UI<br/>React 18 + Vite + TS]
    end

    subgraph CDN["AWS Edge"]
        CF[CloudFront<br/>E3QCDJPG0LCO7E]
        S3[(S3<br/>vip-admin-ui-assets-165505826690)]
        CF --> S3
    end

    subgraph Auth["Auth"]
        COG[Cognito User Pool<br/>us-east-1_MeEkWO4P4]
    end

    subgraph Lambdas["Lambda Functions (Python 3.12)"]
        APL[api-plans<br/>1024MB, 5min, VPC]
        APC[api-campaigns]
        APM[api-metrics]
        APP[api-profiles]
        APS[api-segments]
    end

    subgraph Layer["Lambda Layer"]
        SH[vip_shared<br/>StructuredLogger<br/>OutboundCampaignsClient<br/>RedisLeadSource<br/
>CustomerProfilesClient<br/>audit, filter, schedule, ...]
    end

    subgraph Data["Data Stores"]
        DDB[(DynamoDB<br/>VipAdminPlans)]
        REDIS[(ElastiCache Redis<br/>wait_list:{team}:list)]
        AUDIT[(DynamoDB<br/>VipAdminAudit)]
    end

    subgraph External["External AWS Services"]
        CC[Amazon Connect<br/>Campaigns V2]
        CP[Customer Profiles]
        SNS[(SNS<br/>vip-plans-alerts)]
        EB[EventBridge<br/>Scheduler]
        CW[CloudWatch<br/>Logs + Metrics]
        KMS[KMS CMK]
    end

    UI --> CF
    UI --> COG
    UI -->|REST + JWT| APL
    UI --> APC
    UI --> APM
    UI --> APP
    UI --> APS

    APL --> SH
    APC --> SH
    APM --> SH
    APP --> SH
    APS --> SH

    APL --> DDB
    APL --> REDIS
    APL --> CC
```

```
    APL --> CP
    APL --> SNS
    APL --> EB
    EB --> APL

    APC --> CC
    APP --> CP
    APS --> CP
    APS --> REDIS
    APM --> AUDIT
    APM --> CW

    SH --> CW
    SH --> AUDIT

    DDB -.encrypt.-> KMS
    REDIS -.encrypt.-> KMS
```

2. C4 Component Diagram — api-plans

Key file roles:

File	Role
`handler.py`	Lambda entry point. Distinguishes HTTP invocation (API Gateway shape) from EventBridge invocation (`detail-type` shape).
`router.py`	Maps method + path to handler module functions.
`handlers/plans.py`	CRUD over Plan META records.
`handlers/runs.py`	Run lifecycle: `trigger_run`, `get_run`, `list_runs`, `abort_run`, `apply_plan_to_run`, `force-finish`.
`executor.py`	The orchestration brain. Implements `start_run`, `tick`, `_prestart_plan`, `_dispatch_ready_campaigns`, `_advance_bucket`, `_expire_bucket`, `_complete_run`, `start_run_chained`.
`builders.py`	Pure-function builders: segment names, segment group filters, campaign parameter dicts, flow ARN resolution (`campaign-{STATE}` and `Test-Journey-Flow`).
`store.py`	DynamoDB persistence layer for `VipAdminPlans`. Owns optimistic locking via `_version`.
`scheduler_manager.py`	Wraps EventBridge `PutRule` / `PutTargets` / `DeleteRule` calls; rule naming `vip-plan-{runId}-{bucketIndex}`.

3. State Machine

```
stateDiagram-v2
    [*] --> RunRunning : start_run

    state "Run.status" as RunState {
        RunRunning : running
        RunCompleted : completed
        RunCancelled : cancelled
        RunFailed : failed

        RunRunning --> RunCompleted : all buckets terminal\n_complete_run
        RunRunning --> RunCancelled : abort_run\nor 19:00 COT cutoff
        RunRunning --> RunFailed : unrecoverable executor error
    }

    state "Bucket.status" as BucketState {
        Queued --> Warming : prestart_next AND in pre-warm window
        Queued --> Running : _start_bucket / _activate_warming_bucket
        Warming --> Running : trigger time reached
        Running --> Completed : all campaigns terminal AND advance_bucket
        Running --> Expired : run_duration_minutes exceeded
        Running --> Cancelled : abort_run
        Running --> Error : dispatch raised
        Warming --> Cancelled : abort_run during pre-warm
    }

    state "Campaign.status" as CampaignState {
        CQueued --> CCreating : _start_one_campaign
        CQueued --> CWarming : _prestart_plan creates+starts
        CCreating --> CRunning : Connect returns RUNNING
        CWarming --> CRunning : _activate_warming_bucket sees pre-warmed
        CRunning --> CCompleted : Connect returns COMPLETED
        CRunning --> CCancelled : StopCampaign (abort or 19:00)
        CRunning --> CExpired : bucket expired (bucket_expired) or cutoff_too_close
        CCreating --> CError : CreateCampaign / StartCampaign failed
        CRunning --> CError : poll returned ERROR
        CQueued --> CCompleted : skipped_empty (Redis lookup empty after reconcile)
    }
```

Guards / invariants:

- A run transitions to terminal only when **every** bucket is terminal.
- A bucket transitions to `completed` only when **every** campaign is terminal.
- `_version` MUST increase on every `save_run`; concurrent writes are rejected (`ConcurrentWriteError`).
- `runLock` on the plan METADATA prevents two simultaneous runs of the same plan.
- The 19:00 COT cutoff is checked inside `tick()` and overrides every other transition.

---

4. Pre-warm Flow

Why it matters: Connect V2 enforces `startTime >= now + 6m`. Without pre-warm, a campaign triggered at 08:30 starts dialling at 08:36, losing 6 minutes per bucket. Pre-warm reclaims that.

---

5. Architecture Decision Records

ADR-001 — Amazon Connect V2, segment-driven

Status: Accepted. **Context:** Connect Campaigns offers two ingestion modes: external push (`PutDialRequestBatch`) and segment-driven (a Customer Profiles segment ARN attached to the campaign). External push was attempted in prototype; Connect returned `Operation is not valid` at `StartCampaign`. The instance is provisioned with segment-driven dialing. **Decision:** All campaigns are created with a `source.customerProfilesSegment.segmentDefinitionArn`. Lead lists in Redis are translated into Customer Profiles segments at runtime. **Consequences:** A new segment definition is created per campaign per run (timestamp-encoded name). Segment garbage collection is required eventually (see TD-013).

ADR-002 — DynamoDB single-table, optimistic locking via `_version`

Status: Accepted. **Context:** The two access patterns — "get plan by id" and "list runs for plan" — share the same partition. Concurrent writes (tick + abort, tick + prestart) must not lose data. **Decision:** PK=`PLAN#{planId}`, SK=`META` or `RUN#…`. Writes to runs use `ConditionExpression="attribute_not_exists(#v) OR #v = :current_v"` so the very first write succeeds and every subsequent one rejects when stale. **Consequences:** Callers must catch `ConcurrentWriteError` and either retry or bail; ticks bail because the winning writer already applied state.

ADR-003 — EventBridge Scheduler, one rule per active bucket

Status: Accepted. **Context:** Earlier designs had one rule per run; that didn't model parallel buckets. A rule per campaign was too many rules. **Decision:** Each running or warming bucket has its own rule `vip-plan-{runId}-{bucketIndex}`. Created on bucket activation, deleted on bucket terminal. **Consequences:** Rule cleanup is critical; orphan rules fire ticks against completed runs and silently waste compute. A daily janitor task is on the backlog.

ADR-004 — COT is UTC-5, no DST, computed without pytz

Status: Accepted. **Context:** Colombia does not observe DST. Using `pytz` / `zoneinfo` with the wrong assumptions caused a 1-hour drift in testing. **Decision:** All COT calculations use `datetime.now(timezone.utc) + timedelta(hours=-5)`. The `_DAILY_CUTOFF_HOUR = 19` constant lives in `executor.py`. **Consequences:** If Colombia ever adopts DST this constant breaks. No current risk.

ADR-005 — SNS topic imported by ARN, not CDK-managed

Status: Accepted. **Context:** The CDK execution role inside the `EngineeringPermissionBoundary` cannot call `SNS:CreateTopic` or `SNS:GetTopicAttributes`. CDK deploys failed at synth-time. **Decision:** Topic is created manually once. CDK uses `sns.Topic.fromTopicArn` to import it. The execution role still gets `sns:Publish` granted. **Consequences:** Subscriptions are managed via console/CLI, not IaC. They are listed in `INTEGRATION_CONTRACTS.md`.

ADR-006 — Shared Lambda Layer (`vip_shared`)

Status: Accepted. **Context:** Five api- Lambdas share the same StructuredLogger, OutboundCampaignsClient, RedisLeadSource, audit, filter evaluator, schedule evaluator, and Customer Profiles client. Inlining duplicates code and bloats deployment artifacts. Decision: Publish a single Lambda Layer containing *`vip_shared/…`* under *`python/`*. Each Lambda imports *`from vip_shared.infrastructure...`*. Consequences:* Layer version pinning is the source of truth for shared behaviour. A layer bump must be followed by a Lambda code update to pick it up.

ADR-007 — Pre-warm via `pendingWarmup` on the plan record

Status: Accepted. **Context:** Connect V2 enforces `startTime >= now + 6m`. Without pre-warm, every time-triggered bucket starts dialling 6 minutes late. **Decision:** A separate `prestart_check` EventBridge rule scans plans for triggers 4–6 minutes from now and pre-creates + pre-starts stage-1 campaigns. The pre-warmed identifiers are written to `pendingWarmup` on the Plan META. `start_run` consumes and clears it. **Consequences:** Two writes to the Plan record per pre-warm cycle, partial-warmup recovery on retry, and cleanup of `pendingWarmup` on abort.

ADR-008 — Flow ARN resolved by name convention

Status: Accepted. **Context:** Hardcoding contact-flow ARNs across environments is brittle. Operator-facing UI shouldn't expose ARNs. **Decision:** `builders.resolve_campaign_flow_arn(state_code)` looks up `campaign-{STATE}` (e.g. `campaign-NY`) via `connect:ListContactFlows`. If missing, the function auto-creates a canonical CAMPAIGN-typed flow. `resolve_journey_flow_arn()` looks up `Test-Journey-Flow` (singular; same flow for all states). **Consequences:** Renaming a flow in the Connect console breaks campaigns. Auto-creation requires `connect:CreateContactFlow` IAM (granted).

---

6. Technical Debt Register

ID	Title	Impact	Effort	Notes
TD-001	No staging environment	HIGH	HIGH	Production is the only environment. Smoke tests rely on real traffic.
TD-002	stdlib `logger.*` invisible in CloudWatch	MEDIUM	MEDIUM	`_slog` (StructuredLogger) works; legacy `logger.info(...)` lines emit nothing useful. Convert remaining call sites.
TD-003	No frontend tests	MEDIUM	HIGH	Vitest + Testing Library scaffolding required.
TD-004	ESLint not configured in frontend	LOW	LOW	Add `eslint.config.mjs`, plug into `npm run lint`.
TD-005	`reconcileRetryLimit=1` legacy snapshots	MEDIUM	LOW	Either bump on read or run a one-time migration script.

ID	Title	Impact	Effort	Notes
				Currently surfaces as false `skipped_empty`.
TD-006	Single-account deployment	HIGH	HIGH	Medwork HIPAA policy mandates dedicated account per workload. Inherited shared account.
TD-007	No CI/CD pipeline	HIGH	MEDIUM	All deploys are manual `deploy.sh` runs.
TD-008	No integration tests	MEDIUM	HIGH	All 262 tests are pure unit tests with `boto3` stubbed.
TD-009	No HTTP-layer tests for `handlers/plans.py` or `handlers/runs.py`	MEDIUM	LOW	Coverage stops at executor + store.
TD-010	Compiled `*.js` / `*.d.ts` in `infra/` untracked / partly committed	LOW	LOW	Update `.gitignore`.
TD-011	`startTime = now + 6m` permanent on pre-warm failure	MEDIUM	MEDIUM	Recovery requires next bucket pre-warm cycle.
TD-012	No canary / blue-green Lambda deploy	MEDIUM	HIGH	Hard-cutover `update-function-code`. CodeDeploy alias-shifting would close this.
TD-013	Customer Profiles segment definitions never garbage-collected	LOW	LOW	Each run creates 1 segment per campaign. Slow leak; add a daily janitor.
TD-014	System `zip` broken on dev host	LOW	LOW	`deploy.sh` uses Python `zipfile` workaround; document in onboarding.